08 — CI/CD

Key Concepts

Term	Definition
CI (Continuous Integration)	Developers merge code frequently; each merge triggers automated build + tests
Continuous Delivery	Code always in deployable state; human approves final push to production
Continuous Deployment	Every passing build deploys to production automatically
Pipeline	Automated assembly line: build → test → scan → deploy
Feature Flag	External toggle to enable/disable code without redeploying
Deploy	Code exists in production — may not be visible to users
Release	Feature flag turned on — users can now see the feature
Canary Release	Gradual rollout to small % of users before full deployment

CI/CD Pipeline Flow

```
Developer pushes to feature branch
        ↓
   [Build]              ← Does it compile?
        ↓
   [Unit Tests]         ← Does the logic work?
        ↓
   [Integration Tests]  ← Do components connect?
        ↓
   [Security Scan]      ← Any vulnerabilities?
        ↓
   [Deploy to Staging]
        ↓
   [E2E / Functional Tests]
        ↓
   [Deploy to Production]  ← Manual (Delivery) or Automatic (Deployment)
```

Each step is a gate — fail one, pipeline stops. Nothing broken reaches production.

Continuous Delivery vs Continuous Deployment

	Continuous Delivery	Continuous Deployment
Definition	Code always deployable	Every passing build auto-deploys

	Continuous Delivery	Continuous Deployment
Human approval	Yes — for production push	No — fully automated
Risk	Lower	Higher — needs strong test coverage
Example	Most enterprises	Netflix, Facebook

Feature Flags

- Toggle code on/off via external config — no redeployment needed
- New code ships to production with flag OFF — users don't see it
- Release = flip the flag ON
- Enable gradual rollouts: 1% → 10% → 100%
- Tools: **LaunchDarkly**, **AWS AppConfig**, database-based flags

---

Deployment Strategies

1. Blue-Green Deployment

```
Users → [Blue v1]     [Green v2] ← deploy here
              ↓  switch traffic
Users → [Green v2]    [Blue v1]  ← instant rollback available
```

- Zero downtime — traffic switch is instant
- Instant rollback — switch back to Blue
- Cost: double the infrastructure

---

2. Canary Deployment

```
95% users → [v1]
 5% users → [v2]  ← monitor errors and latency
        ↓ if healthy
100% users → [v2]
```

- Real user traffic tests new version before full rollout
- Best for high-risk features
- Used by: Google, Netflix, Facebook

---

3. Rolling Deployment

```
[v1][v1][v1][v1] → [v2][v1][v1][v1] → [v2][v2][v2][v2]
```

- Replace servers one at a time — no extra infrastructure
- Risk: both versions run simultaneously — API must be backwards compatible

---

Deployment Strategy Comparison

Strategy	Downtime	Rollback	Extra Cost	Key Risk
Blue-Green	None	Instant	Double infra	Cost
Canary	None	Gradual	Minimal	Partial exposure to bugs
Rolling	None	Slow	None	API backwards compatibility

Industry Examples

- **Netflix** — thousands of deploys per day via full CD; feature flags control user exposure; auto-rollback if error rates spike
- **Google** — canary deployments for Search updates; 1% of traffic sees new algorithm first
- **Amazon** — blue-green deployments for Prime Day; instant rollback capability when handling millions of concurrent users
- **Facebook** — feature flags on every new feature; gradual rollout from employees → 1% → global

---

Interview Questions

Q1: What is CI/CD? CI means developers merge code frequently to a shared branch, triggering automated build and tests to catch issues early. CD is either Continuous Delivery (human approves production push) or Continuous Deployment (every passing build deploys automatically).

Q2: What is a CI/CD pipeline? An automated assembly line: build → unit tests → integration tests → security scan → staging deploy → E2E tests → production deploy. Each step is a gate — fail one, pipeline stops.

Q3: What is a feature flag and why is it important in CD? An external toggle that enables/disables code without redeploying. In CD, code ships automatically — flags keep new features hidden until ready and enable gradual rollouts.

Q4: What is the difference between deploying and releasing? Deploying = code is in production but may not be visible to users. Releasing = turning on the feature flag so users can see it.

Q5: Compare Blue-Green, Canary, and Rolling deployments. Blue-Green: two environments, instant traffic switch, zero downtime, instant rollback — costs double infrastructure. Canary: small % of users first, monitor, then full rollout — best for high-risk features. Rolling: replace servers one at a time, no extra infrastructure — API must stay backwards compatible.